### And, Or, Not

The and and && operators evaluate their first operand. If false, the expression returns the value of the first operand; otherwise, the expression returns the value of the second operand:

*expr1* and *expr2*
*expr1* && *expr2*

The or and || operators evaluate their first operand. If true, the expression returns the value of their first operand; otherwise, the expression returns the value of the second operand:

*expr1* or *expr2*
*expr1* || *expr2*

The not and ! operators evaluate their operand. If true, the expression returns false. If false, the expression returns true.

The word forms of these operators (and, or, and not) have a lower precedence than the corresponding symbol forms (&&, ||, and !). For details, see Table 13, Ruby operators (high to low precedence), on page 455.

### defined?

The defined? keyword returns nil if its argument, which can be an arbitrary expression, is not defined. Otherwise, it returns a description of that argument. For examples, check out the tutorial on page 151.

### Comparison Operators

Ruby defines the comparison operators ==, ===, <=>, <, <=, >, >=, and =~. All these operators are implemented as methods. By convention, the language also uses the standard methods eql? and equal? (see Table 3, Common comparison operators, on page 152). Although the operators have intuitive meaning, it is up to the classes that implement them to produce meaningful comparison semantics. The module Comparable provides support for implementing the operators ==, <, <=, >, and >=, as well as the method between? in terms of <=>. The operator === is used in case expressions, described in case Expressions, on page 459.

Both == and =~ have negated forms, != and !~. If an object defines these methods, Ruby will call them. Otherwise, a != b is mapped to !(a == b), and a !~ b is mapped to !(a =~ b).

### Ranges in Boolean Expressions

if *expr1* .. *expr2*
while *expr1* .. *expr2*

A range used in a boolean expression acts as a flip-flop... and .... It has two states, set and unset, and is initially unset.

1. For the three-dot form of a range, if the flip-flop is unset and expr1 is true, the flip-flop becomes set and the the flip-flop returns true.
2. If the flip-flop is set, it will return true. However, if expr2 is not true, the flip-flop becomes unset.
3. If the flip-flop is unset, it returns false.

The first step differs for the two-dot form of a range. If the flip-flop is unset and expr1 is true, then Ruby only sets the flip-flop if expr2 is not also true.